Lab Environment and Foundation Overview

Project 3: PowerShell Automation for Active Directory

Author: Mohd. Ismayeel Khan | **Date:** 12 to 14 October 2026 |
Environment: VMware Workstation 17 Player | Windows Server 2022 | Windows PowerShell 5.1 | **Domain:** Project.local

Related projects: Project 1, Active Directory User Management Lab | Project 2, Enterprise IT Support Lab

---

1. Project overview

This is the third project in my IAM and helpdesk portfolio. In Project 1 I built an Active Directory domain and ran the Joiner-Mover-Leaver (JML) lifecycle by hand: creating accounts, moving people between departments and shutting access down when they left. In Project 2 I worked through a helpdesk queue and resolved ten support tickets. Both projects left me with the same question: how do real IT teams avoid doing this work one click at a time?

The answer is automation, so in this project I used PowerShell to build a small toolkit. It creates users from a CSV file, moves them between departments, offboards leavers securely and produces the reports an access review needs. It all runs against the same Project.local domain I built in Project 1.

What I ended up with:

- 8 PowerShell files: a config file, a logging module, a shared-functions module, an environment check and four automation scripts
- 6 CSV files: three sample inputs and three deliberately messy files for testing
- 6 new users onboarded from a CSV file
- 2 users moved between departments
- 2 users offboarded, with the reason and a ticket number recorded on each account
- 4 audit reports, including a test anomaly that the group audit caught

- 3 scripts tested against broken CSV files in dry-run mode

Why it matters. Account work is repetitive, and small mistakes turn into security problems: a typo puts someone in the wrong group, or a forgotten step leaves a leaver's access open. A script does the same steps the same way every time and leaves a record of what it did. I also built in safety rails (dry runs, confirmation prompts and input checks), because automation without them just makes mistakes faster.

How I built this

I want to be upfront about this: I built the project with Claude, an AI assistant, as my coding partner while I learn PowerShell. I don't claim I wrote every line from scratch. What I did myself was set up the lab, run every script, test each stage (including dry runs and broken-data tests), check the results in Active Directory Users and Computers, take the screenshots and write up how everything works. With each project I'm getting better at reading and changing the scripts on my own.

2. What the project covers

Stage	Script	What it does
Joiner	New-BulkUsers.ps1	Creates users from a CSV file, puts each one in the right OU and security group, and sets a random temporary password
Mover	Invoke-Mover.ps1	Moves a user to a new department: adds the new group, removes the old department group, updates the title and moves the account
Leaver	Invoke-Offboarding.ps1	Disables the account, resets the password to a value nobody knows, removes all groups, records the reason and moves the account to Disabled Users
Audit	Get-AuditReports.ps1	Four read-only reports: inactive users, locked-out accounts, password expiry and a group membership audit
Foundation	Config.ps1, Logging-Module.psm1,	Settings, logging, shared helpers and a health check that every script builds on

Stage	Script	What it does
	Shared-Functions.psm1, Test-Environment.ps1	

3. Lab infrastructure and tools

The lab is a single Windows Server 2022 virtual machine acting as the domain controller for Project.local, running in VMware Workstation 17 Player on my own PC. I reused the domain from Project 1, so the OUs and security groups I needed already existed. Everything I used is free or an evaluation licence, so the whole lab cost nothing.

Tool	What I used it for
VMware Workstation 17 Player	Runs the domain controller virtual machine
Windows Server 2022 Standard (evaluation)	Domain controller for Project.local
Active Directory Domain Services	The directory the scripts read and change
Windows PowerShell 5.1 (run as Administrator)	Runs every script
ActiveDirectory PowerShell module	The commands that talk to AD, such as Get-ADUser, New-ADUser and Move-ADObject
Active Directory Users and Computers (ADUC)	A visual second opinion: I checked PowerShell's changes in ADUC as well
CSV files	Input data (new hires, movers, leavers) and report output
Claude (AI assistant)	Coding partner for the scripts

Starting point: what already existed in Active Directory

The config file needs real names, not guesses, so my first step was to ask Active Directory what it actually contains. I listed every OU and every security group (leaving out the built-in ones). This is what the Project 1 domain already had:

OU	Security group
IT	IT-Team
Finance	Finance-Team
HR	HR-Team
Management	Management-Team
Disabled Users	None (this OU holds offboarded accounts)

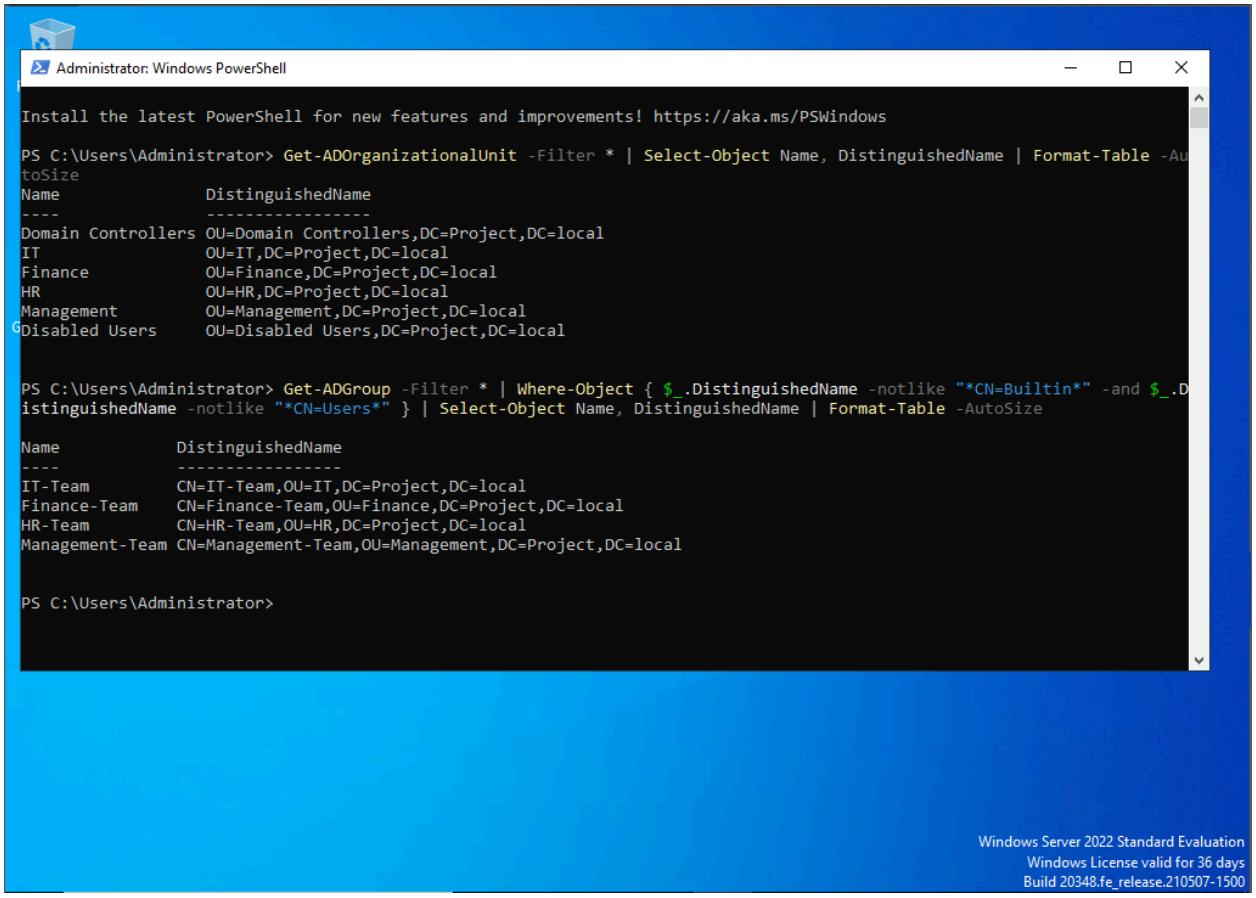


Figure 1: The real OUs and security groups in Project.local, listed before writing any configuration.

The top command lists the OUs and the bottom command lists the security groups. These are the names the config file is built from.

4. How the pieces fit together

I think of the project as a small workshop:

- **Config.ps1 is the address book.** It says which OU and security group each department uses, so no script has OU names typed inside it.
- **Logging-Module.psm1 is the notebook.** Every script writes down what it did, with a timestamp, on screen and in a log file.
- **Shared-Functions.psm1 is the toolbox.** It holds the random password generator, the CSV and OU checks, and a helper that turns AD error messages into readable sentences. I added it in Checkpoint 5 so no code is duplicated.
- **Test-Environment.ps1 is the health check** I run before anything else.
- **Four scripts do the work:** New-BulkUsers.ps1 (Joiner), Invoke-Mover.ps1 (Mover), Invoke-Offboarding.ps1 (Leaver) and Get-AuditReports.ps1 (the read-only audit).

Every automation script follows the same flow:

1. Load the config, logging and shared functions.
2. Log who ran the script, on which machine, and whether it is a live run or a dry run.
3. Check the input before touching anything: the CSV exists, has the right columns, and the OUs it needs exist.
4. Go through the file row by row, skipping and explaining any bad row.
5. Make the change in AD, or in dry-run mode only say what would happen.
6. Write a CSV report and print a summary table.

Architecture: how the pieces fit together

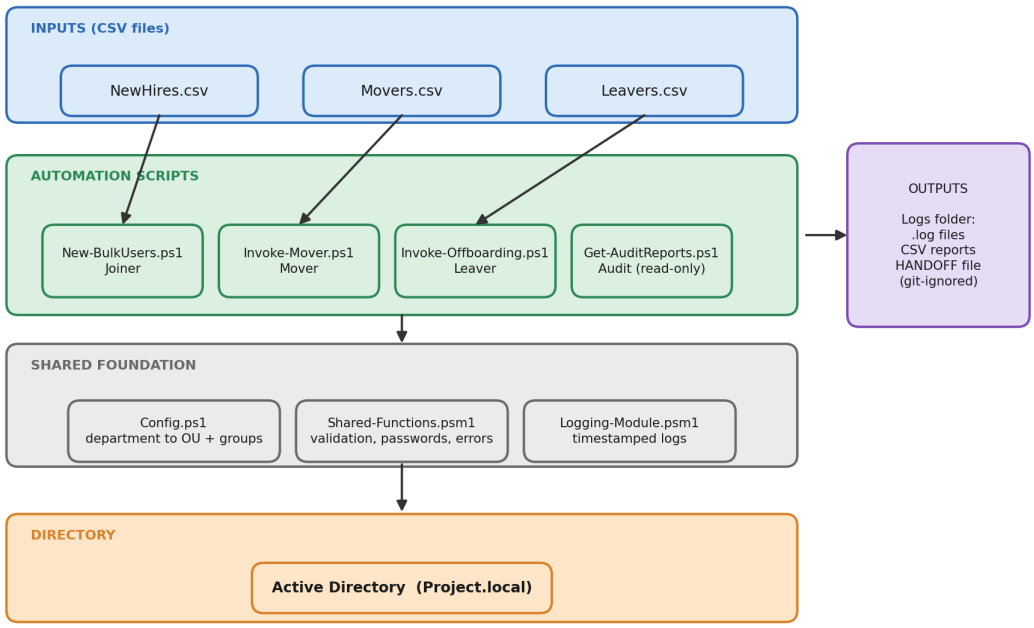


Figure 2: Architecture. CSV inputs feed the scripts, the scripts rely on a shared foundation, and everything ends up in Active Directory and the Logs folder.

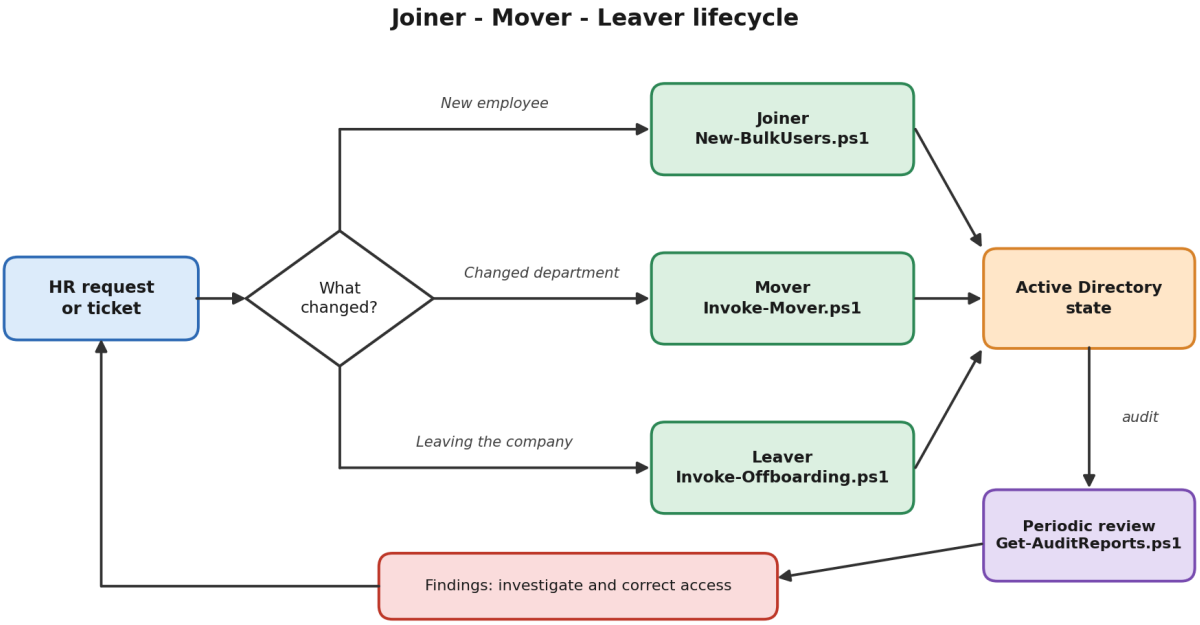


Figure 3: The Joiner-Mover-Leaver lifecycle. A request from HR goes to the right script, and the audit reports feed corrections back into the process.

5. Checkpoint 1: building the foundation

Before automating anything I wanted a base I could trust. A script that guesses at OU names or fails silently is worse than doing the job by hand, so Checkpoint 1 was about building the foundation and proving that it worked.

What I did:

1. Started the domain controller VM and logged in as Administrator.
2. Opened Windows PowerShell as Administrator.
3. Ran two discovery commands to list the real OUs and security groups (Figure 1).
4. Created the project folder at C:\PowerShell-AD-Automation with five subfolders: Scripts, CSV, Logs, Docs and Screenshots.
5. Created the first three files inside the Scripts folder.

The three files:

- **Config.ps1** maps each department (IT, Finance, HR and Management) to its OU path, its security group and the length of the temporary password. If I move this project to another domain, this is the only file I have to change.
- **Logging-Module.psm1** gives every script a Write-Log function that prints coloured, timestamped lines (cyan for information, green for success, yellow for warnings, red for errors) and saves them to a log file.
- **Test-Environment.ps1** checks that PowerShell can reach the domain and that everything named in Config.ps1 really exists in AD.

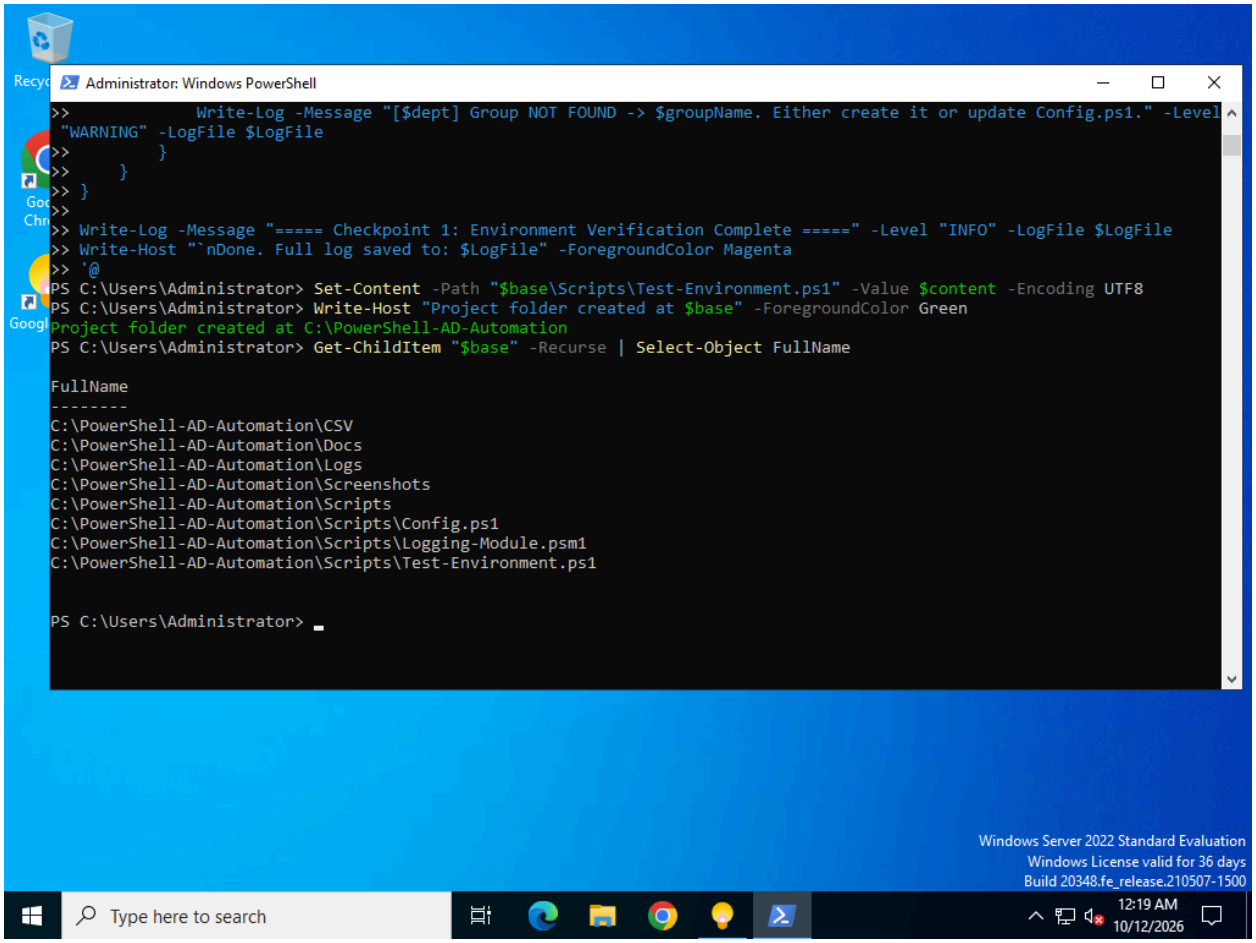


Figure 4: The project folder and script files created on the domain controller.

The listing shows the five folders (CSV, Docs, Logs, Screenshots and Scripts) and the three script files inside Scripts.

6. Verifying the environment

I ran the check from the Scripts folder:

```
cd C:\PowerShell-AD-Automation\Scripts
.\Test-Environment.ps1
```

The script does five things: it loads the ActiveDirectory module, confirms it can reach the Project.local domain, lists every OU and security group in the domain, checks the OU and group of each department in the config, and writes everything to a log file in the Logs folder.

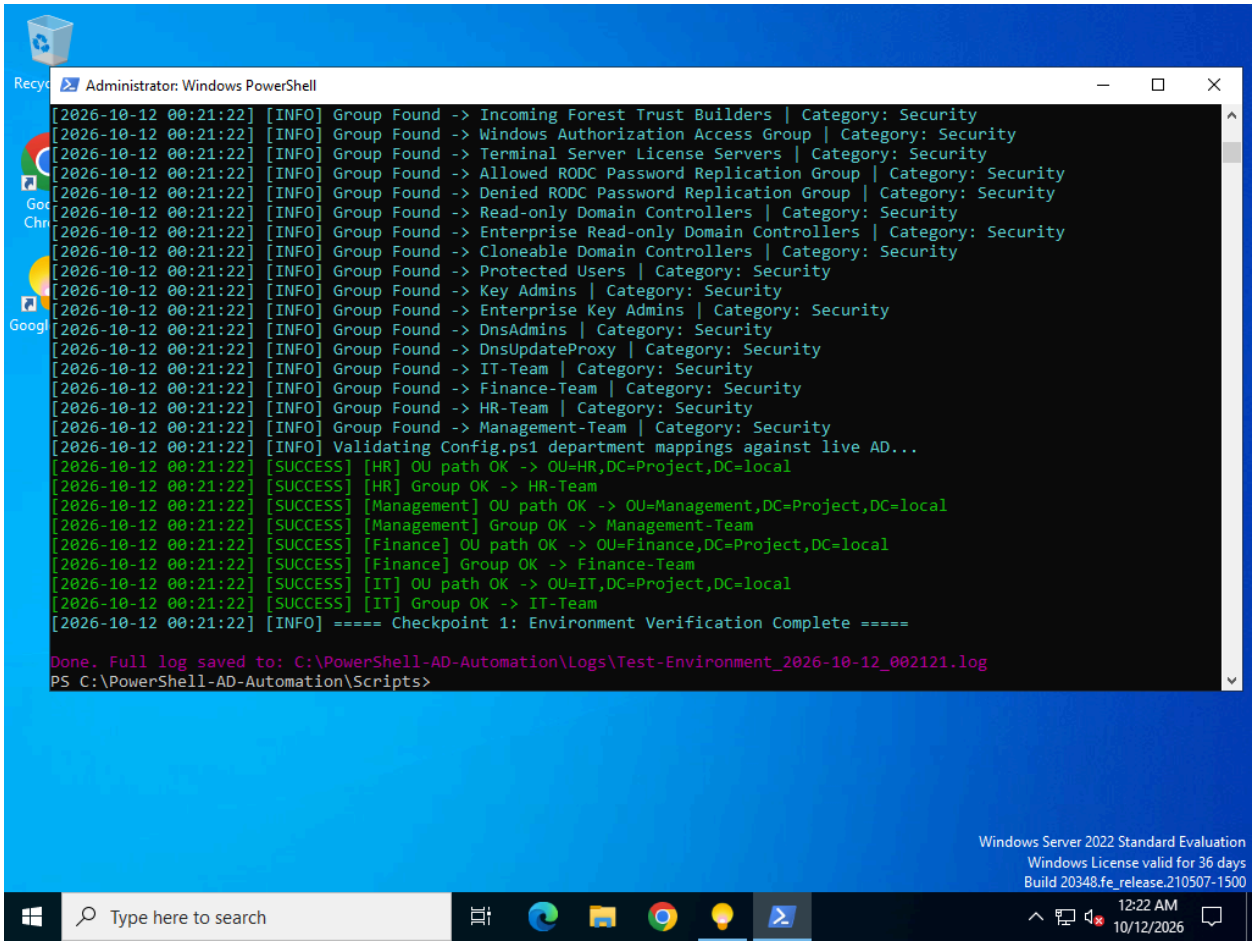


Figure 5: Environment test. Every department's OU and group was validated against live Active Directory.

Result: all four departments (HR, Management, Finance and IT) show OU path OK and Group OK in green. The long run of group names above those lines is normal. The script lists every group in the domain, including the built-in ones, and then checks only the four that matter.

Why this step matters. If a department's OU path in the config were wrong, a later script could create users in the wrong place or fail halfway through a batch. Catching that here, before any account is touched, is far cheaper than fixing it afterwards.

7. Safety and security design

These are the rules I built into the scripts, and each one has its own evidence in the other documents:

- **Dry run first.** Every script supports -WhatIf, which shows exactly what would happen and changes nothing.
- **Confirmation before destructive actions.** Offboarding stops and asks Yes or No for every user.
- **Disable, never delete.** A leaver's account is disabled and kept, so it is available for audits and recovery.
- **Protected accounts.** Administrator, Guest, krbtgt and members of Domain Admins are refused by the Leaver and Mover scripts.
- **Add before remove.** When someone changes department, the new access is added first so there is no gap, and then the old department access is removed.
- **Input validation.** Bad CSV rows are skipped with a clear reason and never stop the good rows.
- **Safe to re-run.** Running a script twice does not create duplicates or repeat work.
- **Audit trail.** Every run logs who ran it, where, and in which mode, and every action ends up in a CSV report.
- **Passwords kept out of reports.** Temporary passwords go to a separate hand-off file that is never shown in a report or uploaded to GitHub.
- **Least privilege.** Movers lose their old department access, and leavers lose all group membership.

8. Project timeline

Checkpoint	Date	What I did	Outcome
1. Foundation	12 Oct 2026	Config, logging and environment check	All four departments validated against live AD
2. Joiner	12 Oct 2026	Onboarding script and sample CSV	6 users created in the right OUs and groups
3. Leaver	12 Oct 2026	Offboarding script with confirmation prompt	2 users offboarded with reason and ticket recorded

Checkpoint	Date	What I did	Outcome
4. Mover and audit	14 Oct 2026	Mover script and four audit reports	2 users moved, anomaly detected by the group audit
5. Validation	14 Oct 2026	Shared module, input checks, error handling	3 scripts tested with broken CSV files
6. Documentation	October 2026	Documents and GitHub repository	This document set

9. Skills demonstrated

- Active Directory administration: users, OUs, security groups and account lifecycle
- Joiner-Mover-Leaver (JML) workflows and role-based access
- PowerShell scripting: CSV processing, functions, modules, error handling and logging
- Least privilege and prevention of privilege creep
- Secure offboarding: disable, reset, remove access, retain and document
- Access review reporting: inactive accounts, lockouts, password expiry and group audits
- Input validation and safe automation (dry runs, confirmations, idempotent scripts)
- Technical documentation and evidence collection

10. Where to read next

- **Document 2, JML Operations Log:** the Joiner, Leaver and Mover runs, step by step, with screenshots.
- **Document 3, Audit and Validation Report:** the four audit reports and the tests with deliberately broken CSV files.
- **Document 4, Runbook and Script Reference:** a reusable guide for running each script.